MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 12
Penyimpanan dan Pembacaan Data ke File



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P12
Penyimpanan dan Pembacaan Data ke File

Mata Kuliah: Praktikum Pemrograman Berorientasi Obyek
Model Pembelajaran: Project Based Learning
Prasyarat: Collection pada P11 dan exception handling pada P10.

1. Tujuan Praktikum

Mahasiswa mampu:

- menyimpan data collection ke file teks;

• membaca file dan membentuk kembali objek;
• menangani kegagalan operasi file;
• memvalidasi data sebelum mengganti collection.

2. Dasar Teori Singkat

Objek dalam collection berada di memori selama program berjalan. Agar tersedia setelah program ditutup, datanya perlu disimpan pada media penyimpanan. Proses ini disebut **persistensi data**.

Pada praktikum ini, gunakan file teks dengan format **satu buku per baris**: `B001|Dasar Java`
`B002|Konsep OOP`

Karakter | memisahkan kode dan judul.

Komponen	Fungsi
Path	Merepresentasikan lokasi file.
Files.write()	Menulis data ke file.
Files.readAllLines()	Membaca seluruh baris file.
StandardCharsets.UTF_8	Menentukan encoding teks.
IOException	Menyampaikan kegagalan operasi baca/tulis.

Java menyediakan operasi tersebut melalui java.nio.file.Files. Pembacaan seluruh baris sesuai untuk file kecil pada latihan ini. docs.oracle.com

Prinsip utama: data hasil pembacaan harus diperiksa sebelum menggantikan collection yang sedang digunakan.

3. Studi Kasus

Kembangkan program perpustakaan P11 dengan dua fitur:

1. **Simpan:** menulis seluruh koleksi buku ke buku.txt.
2. **Muat:** membaca buku.txt menjadi kumpulan objek Buku.

Aturan:
• kode dan judul tidak boleh kosong;
• kode buku harus unik;
• kode dan judul tidak boleh mengandung | atau pergantian baris;
• jika pemuatan gagal, collection sebelumnya tetap dipertahankan.

4. Langkah Praktikum

Langkah 1 — Siapkan program P11

Gunakan kembali class Buku dan Perpustakaan dari P11. Tambahkan import berikut di bagian paling atas file:

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
```

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
```

Import ArrayList dan List dari P11 tetap digunakan.

Langkah 2 — Tambahkan method penyimpanan

Letakkan method berikut **di dalam class Perpustakaan**, sebelum kurung penutup class.

```
public void simpan(Path lokasi) throws IOException {
 List<String> baris = new ArrayList<>();

 for (Buku buku : daftarBuku) {
 String kode = buku.getKode();
 String judul = buku.getJudul();

 for (String nilai : new String[]{kode, judul}) {  if
(nilai.contains("|")
 || nilai.contains("\n")
                    || nilai.contains("\r")) {
 throw new IllegalArgumentException(
 "Kode/judul mengandung pemisah terlarang."  );
 }
 }

 baris.add(kode + "|" + judul);
 }

 Files.write(lokasi, baris, StandardCharsets.UTF_8);
}
```

Seluruh data diperiksa sebelum penulisan dimulai.

Perhatikan: pemanggilan Files.write() ini membuat file jika belum ada dan menimpa isinya jika sudah ada. docs.oracle.com

Langkah 3 — Tambahkan method pembacaan

Tambahkan method berikut di dalam class yang sama:

```
public void muat(Path lokasi) throws IOException {
 List<String> baris = Files.readAllLines(
 lokasi, StandardCharsets.UTF_8
 );

 Perpustakaan sementara = new Perpustakaan();

 for (int i = 0; i < baris.size(); i++) {
 String[] bagian = baris.get(i).split("\\|", -1);

 if (bagian.length != 2) {
 throw new IOException(
 "Format salah pada baris " + (i + 1)
 );
```

```
 }

 try {
 Buku buku = new Buku(bagian[0], bagian[1]);
sementara.tambahBuku(buku);

 } catch (IllegalArgumentException e) {
 throw new IOException(
 "Data tidak valid pada baris " + (i + 1)  + ": " +
e.getMessage(),
 e
 );
 }
 }

 // Ganti collection hanya setelah semua baris valid.
daftarBuku.clear();
 daftarBuku.addAll(sementara.daftarBuku);
}
```

Penjelasan:

- split("\\|", -1) memisahkan data dan mempertahankan kolom kosong agar dapat divalidasi.
- Constructor Buku memeriksa kode dan judul.
- tambahBuku() dari P11 memeriksa kode duplikat.
- Objek sementara menampung hasil pembacaan sebelum digunakan. •

File kosong dianggap sebagai koleksi kosong yang valid.

Langkah 4 — Ganti class Main

Hapus class Main lama, lalu gunakan:

```
public class Main {
 public static void main(String[] args) {
 if (args.length != 1
 || !(args[0].equals("simpan")
 || args[0].equals("muat"))) {
 System.out.println(
 "Gunakan: java Main simpan atau java Main muat"  );
 return;
 }

 Perpustakaan perpustakaan = new Perpustakaan();  Path
lokasi = Paths.get("buku.txt");

 try {
 if (args[0].equals("simpan")) {
 perpustakaan.tambahBuku(
 new Buku("B001", "Dasar Java")
 );
```

```
perpustakaan.tambahBuku(
new Buku("B002", "Konsep OOP")
);

perpustakaan.simpan(lokasi);
System.out.println("Data berhasil disimpan.");

} else {
perpustakaan.muat(lokasi);
perpustakaan.tampilkanBuku();
}

} catch (IOException e) {
System.out.println(
"Operasi file gagal: " + e.getMessage()
);

} catch (IllegalArgumentException e) {
System.out.println(
"Data ditolak: " + e.getMessage()
);
}
}
}
```

Langkah 5 — Jalankan dalam dua proses berbeda

Kompilasi dan simpan:

```
javac Main.java
java Main simpan
```

Setelah program selesai, jalankan kembali untuk membaca:

```
java Main muat
```

Hasil yang diharapkan:

```
B001 - Dasar Java
B002 - Konsep OOP
```

Total buku: 2

File buku.txt berada di **direktori kerja saat program dijalankan**. Gunakan direktori yang sama untuk kedua perintah.

5. Pengujian

Cadangkan buku.txt sebelum mengubah isinya untuk pengujian.

Skenario	**Hasil yang diharapkan**
Simpan dua buku valid	File berisi dua baris data.
Tutup program, lalu jalankan mode muat	Kedua objek terbentuk kembali.
Muat file yang tidak ada	Pesan kegagalan operasi file muncul.

Ubah satu baris menjadi B003	Ditolak karena format salah.
Ubah satu baris menjadi `B003	`
Isi file dengan kode duplikat	Pemuatan ditolak.
Muat file kosong	Collection menjadi kosong.

Uji konsistensi state: isi collection dengan satu buku terlebih dahulu, lalu panggil muat() terhadap file yang rusak. Tangkap exception dan tampilkan collection. Buku sebelumnya harus tetap tersedia.

6. Tugas Proyek Kelompok

Lanjutkan proyek P11 dengan ketentuan:

1. Tambahkan fitur **simpan dan muat** untuk minimal satu collection objek.
2. Tentukan format file dan arti setiap kolom.
3. Terapkan kembali validasi domain saat membentuk objek dari file.
4. Tangani file tidak ditemukan, format salah, dan data tidak valid.
5. Buktikan data dapat dimuat setelah program dijalankan ulang.
6. Pastikan kegagalan pemuatan tidak menghapus data yang sedang digunakan. Pada contoh awal, operasi file ditempatkan di Perpustakaan agar alurnya mudah dipelajari. Untuk pengembangan proyek, pisahkan tanggung jawab tersebut ke class seperti **BukuFileRepository**.

Batasan contoh: penyimpanan langsung ini belum menjamin file lama tetap utuh jika terjadi kegagalan saat penulisan. Penyimpanan melalui file sementara dapat dijadikan pengembangan tambahan.

7. Luaran dan Penilaian

Kumpulkan **source code, contoh file data, screenshot simpan–muat, serta tabel hasil pengujian**.

Aspek	**Bobot**
Ketepatan penyimpanan data	25%
Pembacaan dan pembentukan kembali objek	25%
Validasi dan konsistensi collection	25%
Penanganan kegagalan operasi file	15%
Pengujian dan penjelasan kode	10%
Total	**100%**

8. Pertanyaan Evaluasi

1. Apa perbedaan data di collection dan data di file?
2. Mengapa data dari file tetap harus divalidasi?
3. Apa risiko mengosongkan collection sebelum seluruh file selesai diperiksa?
4. Mengapa fitur simpan harus dipanggil kembali setelah data diubah?

Tugas minggu depan: integrasikan fitur tambah, cari, hapus, simpan, dan muat dalam menu aplikasi proyek. Uji satu alur lengkap mulai dari memasukkan data hingga memuatnya pada eksekusi berikutnya.